A Formal Software Engineering Methodology for Semantically Aligned Digital Twins

Name, Last Name
University
University
{name.last_name}@uni.com
City, Country

***Abstract*—In safety-critical cyber-physical systems (CPS), digital twins (DTs) are used alongside their physical counterparts to support monitoring, prediction, and decision-making. Driven by changing regulations, design improvements, and emerging optimization opportunities, both the physical twin (PT) and its digital counterpart evolve throughout development and operation. Existing frameworks typically assume alignment between PT and DT without providing formal guarantees. When PT and DT models are developed independently using different abstractions and vocabularies, maintaining alignment becomes challenging, undermining the reliability of DT-based reasoning. This paper introduces a formal framework based on model-based systems engineering (MBSE) for reasoning about semantic alignment between PT and DT. We define semantic alignment as consistency in the interpretation of data and behavior matching between PT and DT models with respect to a shared ontology capturing domain knowledge. We show that this alignment enables the transfer of behavioral and data properties between PT and DT models, and that these guarantees extend to deployed systems and are preserved under ontology refinement**

***Index Terms*—digital twins, semantic alignment, ontologies**

I. Introduction

The development of safety-critical cyber-physical systems (CPS) follows an iterative process in which requirements, models, and implementations co-evolve across design cycles, driven by changing regulations, design improvements, and emerging optimization opportunities. Model-based systems engineering (MBSE) [1] is the established discipline for managing this complexity in safety-critical industries such as automotive and aerospace [2], [3], [4], [5], providing a rigorous framework of formal models that capture system structure, behavior, and requirements, and through which correctness can be established and preserved across iterations.

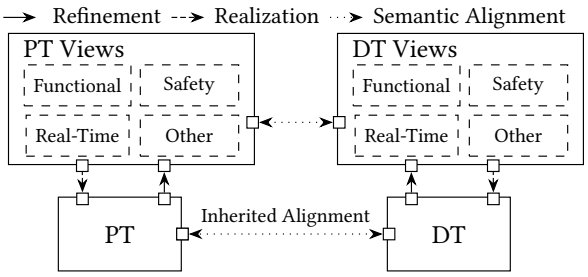
Complementary to MBSE, the concept of Digital Twins (DTs) [6] has emerged as a powerful software engineering paradigm to support decision-making across the lifecycle of CPSs. A DT pairs a virtual representation of the system with a connection to the deployed system, referred to as the Physical Twin (PT), enabling continuous synchronization, run-time monitoring, and decision support throughout the system's operational life [6], [7], [8]. DTs have demonstrated broad applicability across domains [9], [10], and their use has been extended beyond monitoring and predictive analytics to support iterative design refinement, virtual prototyping, and post-deployment adaptation [11], [12], [13].

These properties hinge on the requirement that he meaning assigned to observed data must agree between the two twins, and their observable behaviors must correspond, i.e. that the DT and PT remain semantically aligned across operation and evolution. Absence or loss of either form of alignment undermines the correctness of any DT-based reasoning, as a DT that misinterprets a sensor reading or exhibits behavior inconsistent with the PT cannot be trusted for monitoring or decision-making. This is particularly critical in safety-critical domains, and is further exacerbated in modern federated architectures such as Gaia-X [14], where independently developed DTs must interoperate and reason jointly.

Current approaches to DT-PT alignment, however, address neither requirement fully. Runtime monitors [15] are bounded by finite observations and cannot certify the unobserved behavioral space. Model synchronisation mechanisms [16], [12] ensure structural correspondence but not semantic equivalence. Consistency frameworks [17], [18] check only an explicitly chosen property fragment, leaving the remainder unchecked. Ontology-based matching [19], [20] aligns static data schemas rather than dynamic behavior, meaning DT and PT can conform to the same schema while exhibiting observably different behaviors under it. No existing mechanism therefore establishes, at design time, that the DT and PT agree on the meaning of observed data and on their observable behavior, nor guarantee that this agreement is preserved as both evolve across design iterations and operational use.

To bridge this gap, we present a formal software engineering framework based on MBSE for semantically aligned DTs. We formally characterize semantic alignment via a shared ontology between PT and DT, establish it once and compositionally at design time, and derive conditions under which it is preserved across iterative co-evolution. We provide an algorithm to check alignment, and prove that runtime monitors and ontology-based matchers are fundamentally incomplete approximations of this condition. The framework delivers these guarantees at no additional engineering cost: the ontology is already required for Gaia-X compliance, and a single design-time verification replaces the indefinite runtime overhead of monitor instrumentation, deployment, and maintenance.

Fig. 1. Multi-view Architectural Foundation



II. Multi-View Framework

Our proposed software engineering methodology for semantically aligned DTs is grounded in a multi-view framework depicted in Figure 1. In this context, a system view, or PT-view, is a formal model that captures a specific concern of the PT such as timing, functional, or safety constraints through a formal model. By separating concerns into views, the framework enables concern-specific reasoning while maintaining consistency across abstraction levels. Any implementation of the PT must therefore refine these views; that is, exhibit only behaviors permitted by them, preserving all their properties.

The DT is the artifact that enables prediction, monitoring, optimization, and adaptation throughout the PT's operational lifetime. The development of the DT is guided by the DT-specific views, which capture the subset of the corresponding PT-view relevant to the aforementioned tasks. Multiple DT views may correspond to a single PT view and each must be shown to be a refinement of it, ensuring that they faithfully capture it, to ensure the specification-level coherence of the DT with respect to the PT. Similarly to the PT, the DT must also be shown to refine its DT-specific views.

We aim at establishing semantic alignment between the DT and the PT at the view level, rather than at the implementation level. Views are the authoritative design-time artifacts in MBSE and the natural locus of formal reasoning: implementations are generated from or refine them by construction, so view-level alignment propagates to the deployed system without additional proof obligation, independently of implementation technology, vendor, or instance. The multi-view structure further allows each DT-view to be aligned independently with its corresponding PT-view, with overall alignment composed from the parts. Finally, views are abstract and technology-agnostic, and are the artifacts exposed across organizational boundaries in federated settings such as Gaia-X, making view-level alignment the only kind establishable between independently developed twins.

III. Related Work

The concept of DTs was introduced in [6] and later formalized for CPSs [21]. DTs are now widely used as evolving software representations of physical assets driven by real-world data streams [22], [23], [24].

Ontologies and knowledge graphs are widely used in DT architectures for knowledge representation and interoperability [20]. A common approach employs ontologies as schema-level bridges between sensor data and DT models [19], [25]. In contrast, our framework goes beyond structural compatibility and captures behavioral correspondence: semantic alignment requires equivalence of observable behaviors, since twins may conform to the same schema while exhibiting different dynamics. Ontologies are also used as semantic abstractions for cross-domain reasoning, service engineering, and orchestration of DT services [26], [27], [28], [29], [30], [31], [32]. In these approaches, ontologies support conceptual interoperability between heterogeneous artifacts. In our framework, instead, the ontology acts as a formal semantic ground assigning meaning to PT and DT states and labels, enabling alignment to be defined as behavioral equivalence and guaranteeing preservation of temporal properties across all reachable states.

PT/DT alignment is often addressed through runtime monitoring and trace analysis. Munoz et al. [15] detect divergence by aligning PT traces with DT behavior and applying anomaly detection, while Moon et al. [33] verify that observed timed traces conform to a DT model expressed as a Time Colored Petri Net. These approaches rely on syntactic correspondence between traces or executions. In contrast, our framework grounds PT and DT observations in a shared ontology, enabling semantic equivalence even when their observable vocabularies differ. Ontologies are similarly used in [17], [18] to lift DT runtime states into knowledge graphs for semantic monitoring of PT/DT correspondence.

More generally, runtime monitoring approaches establish alignment operationally and over selected constraints, whereas our framework guarantees semantic consistency entirely at design time over all reachable states and all properties expressible over the aligned fragment. This removes the engineering overhead of implementing and maintaining monitoring infrastructures as systems evolve, where changes would require revalidating and updating all monitors. Our framework further supports ontology evolution without invalidating previously established alignment. Runtime monitoring nevertheless remains complementary for detecting deployment-time deviations [24] or quantifying uncertainty under incomplete environment models [9], which lie outside the scope of our work.

Recent work also applies formal methods to DT behavior. Huang et al. [34] use the Temporal Logic of Actions to verify synchronization and information-flow properties of DT architectures, while Liu et al. [35] introduce Semantic Petri Nets that combine RDF/OWL knowledge bases with Petri Net models to validate temporal rules over DT executions. These approaches verify properties of the DT itself, whereas our work formalizes semantic alignment between PT and DT as a behavioral equivalence relation under a shared semantic.

Another line of work studies co-evolution of PT and DT models. Model-driven toolchains propagate structural changes from the PT to maintain DT infrastructure [16], [12]. These approaches ensure structural correspondence between twins but do not address semantic alignment. Our framework is complementary: it does not prescribe how the DT evolves but provides guarantees that the resulting models remain

semantically aligned with respect to domain knowledge.

Taken together, existing work addresses interoperability, runtime monitoring, or verification of DT behavior, but none establishes semantic alignment between PT and DT as a formally verified behavioral equivalence under a shared ontological interpretation.

IV. Preliminaries

The framework introduced in Section II relies on formal models to capture PT and DT views, reason about their behavior, and establish alignment. We now introduce the formal foundations underlying these models: timed transition systems as the semantic domain, timed automata as their syntactic representation, and weak timed bisimulation as the behavioral equivalence relation on which alignment will be grounded.

We model PT and DT views as *Timed Transition Systems (TTS)* [36], [37], the standard semantic model for real-time systems capturing both discrete event-driven behavior and the passage of time. A TTS is defined as follows.

Definition 1. A Timed Transition System is a tuple $\mathcal{T} = (S, \Sigma_E, \Sigma_D, \rightarrow)$, where:

- S is a set of states
- Σ_E is a set of *event labels*,
- Σ_D is a set of *delay labels*, with δ representing the elapse of δ time units.
- $\rightarrow \subseteq S \times (\Sigma_E \cup \Sigma_D) \times S$ is the transition relation.

We make use of *Timed Automaton (TA)* [38], [39] as syntactic representation of each model. A TA is defined as follows.

Definition 2. A Timed Automaton is a tuple $TA = (L, \ell_0, C, \Sigma, E, \text{Inv})$, where:

- L is a finite set of locations with initial location $\ell_0 \in L$,
- C is a finite set of clocks,
- $\Sigma = \Sigma_E \cup \Sigma_D$ is the action alphabet, partitioned into event-driven and delay-driven actions with $\Sigma_E \cap \Sigma_D = \emptyset$,
- $E \subseteq L \times \mathcal{B}(C) \times \Sigma \times 2^C \times L$ is a finite set of edges, where $\mathcal{B}(C)$ denotes clock constraints,
- $\text{Inv} : L \rightarrow \mathcal{B}(C)$ assigns invariants restricting the admissible time elapse in each location.

The semantics of a TA is given as a TTS as of Definition 1, defined as of [38], [39], and written as $[\![TA]\!]$. A *state* of the corresponding TTS is a pair $(\ell, v) \in L \times \mathbb{R}^C_{\geq 0}$, where ℓ is the current location and v a clock valuation. Discrete transitions correspond to edges in E with clock resets, while delay transitions correspond to uniform advancement of all clocks under the invariant of the current location.

To formalize the expected behavior of systems, we use Timed Computation Tree Logic (TCTL) properties [36]. The semantics of a property ϕ, denoted by $[\![\phi]\!]$, is the set of traces that satisfy ϕ. A TA TA satisfies a formula ϕ, written $TA \models \phi$, iff $[\![TA]\!] \subseteq [\![\phi]\!]$. A property φ over an alphabet X is built from labels in X using boolean connectives ($\wedge$, $\vee$, $\neg$) and timed path quantifiers (**A**, **E**, **G**, **F**, **U**); we refer to [36] for the full syntax. Given a function $\sigma : X \rightarrow Y$, we write $\sigma(\varphi)$ for the property obtained by replacing each label a in φ with $\sigma(a)$.

To reason about the behavioral relationship between two TTS, we use weak timed simulation (WTS) and weak timed bisimulation (WTB). Intuitively, a simulation captures the idea that one system can match every observable move of another, while abstracting over internal transitions. Bisimulation strengthens this to a symmetric requirement: neither system can exhibit an observable behaviour the other cannot match.

Formally, internal actions are modeled as τ-transitions, which arise from projection of event labels outside the alphabet of interest. We write $s \overset{\varepsilon}{\Rightarrow} s'$ if s' is reachable from s by any finite sequence of τ-transitions and delay transitions, and $s \overset{a}{\Rightarrow} s'$ for $a \in \Sigma_E \setminus \{\tau\}$ if there exist s_1, s_2 such that $s \overset{\varepsilon}{\Rightarrow} s_1 \overset{a}{\rightarrow} s_2 \overset{\varepsilon}{\Rightarrow} s'$.

Definition 3. A relation $R \subseteq S_1 \times S_2$ is a WTS from $\mathcal{T}_1$ to $\mathcal{T}_2$ on $X \subseteq \Sigma_{E_1} \cap \Sigma_{E_2}$ iff $(s_0^1, s_0^2) \in R$ and for every $(s, t) \in R$:

- $\forall a \in X, s'$ s.t. $s \overset{a}{\Rightarrow} s'$, $\exists t'$ with $t \overset{a}{\Rightarrow} t'$ and $(s', t') \in R$;
- $\forall \delta \in \Sigma_D$ and s' with $s \overset{\delta}{\rightarrow} s'$, $\exists t'$ reachable from t by $\overset{\varepsilon}{\Rightarrow}$ with elapsed time δ such that $(s', t') \in R$.

We write $\mathcal{T}_1 \preccurlyeq \mathcal{T}_2$ iff such a relation exists.

Definition 4. A relation R is a WTB on X iff both R and R^{-1} are WTSs on X. We write $\mathcal{T}_1 \sim \mathcal{T}_2$.

As a standard result, if $\mathcal{T}_1 \sim \mathcal{T}_2$ on alphabet X, then for every TCTL property φ over X: $\mathcal{T}_1 \models \varphi \iff \mathcal{T}_2 \models \varphi$ [37]. If $\mathcal{T}_1 \preccurlyeq \mathcal{T}_2$ on alphabet X, then for every safety property φ over X: $\mathcal{T}_2 \models \varphi \Rightarrow \mathcal{T}_1 \models \varphi$ [37].

V. Formalization of Domain Knowledge

Engineers design systems against an understanding of the application domain, i.e., the entities, relationships, and constraints that govern its behavior. Ontologies are the standard tool for representing this knowledge in DT architectures, typically formalized as knowledge graphs [18], [20], [25]. We generalize this notion to a first-order theory, which subsumes knowledge graphs while supporting richer engineering knowledge and automated reasoning via Satisfiability Modulo Theories (SMT) solvers [40].

Definition 5. An ontology is a tuple $K = (S, F, R, \Delta)$ where:

- S is a finite set of value domains, representing the types of entities in the domain;
- F is a finite set of function symbols with declared value domains signatures, representing measurable quantities and domain-specific computations;
- R is a finite set of relation symbols with declared value domains signatures, representing domain predicates;
- Δ is a finite set of axioms, with $\Delta \subseteq \mathrm{F}(S, F, R)$, where $\mathrm{F}(S, F, R)$ is the set of all first-order sentences over the signature, expressing what the engineer knows to be true and invariant about the domain.

The signature (S, F, R) define the vocabulary of the domain. A model of K is an instantiation of (S, F, R) satisfying Δ.

We write $\mathcal{L}(K)$ for the set of all facts expressible by the vocabulary that are entailed by Δ.

Example 1. Consider a drone used to spray pesticides over a field, powered by an onboard battery. The engineering knowledge can be formalized as $K = (S, F, R, \Delta)$:

- $S = \{Hectare, \mathbb{R}, \mathbb{N}\}$, capturing units of land, real-valued energy, and natural-valued hectare indices.
- $F = \{consumption : Hectare \to \mathbb{R},\ num\_hectares : \mathbb{N},\ total\_consumption : \mathbb{R},\ battery\_capacity : \mathbb{R}\}$, where $consumption(h)$ is the energy needed to cover hectare h, and the remaining symbols are constants.
- $R = \{covered : Hectare\}$, where $covered(h)$ holds when hectare h has been sprayed.
- Δ contains the following axioms: $num\_hectares = 10$, $battery\_capacity = 500$, $\forall h : Hectare.\ consumption(h) \geq 0$, $total\_consumption = \sum_{h\,:\,covered(h)} consumption(h)$.

Several approaches exist for constructing ontologies in this form from system documentation, models, or domain standards, enabling the systematic construction of machine-interpretable domain knowledge [41], [42], [43]. Knowledge graphs themselves admit a natural logical interpretation in which entities correspond to individuals, relations to predicates, and triples to ground atoms. As a result, representing the ontology as a first-order theory builds directly on existing ontology engineering practices and artifacts, while enabling richer constraints and automated reasoning capabilities.

An ontology captures the structure of the application domain independently of any particular system. When in MBSE a formal model is established, engineers draw on this knowledge to choose labels that reflect actuator commands, threshold crossings, or compliance events. This abstraction is motivated by the need for tractable model checking, but this connection is rarely made explicit. The model retains the labels while discarding the reasoning behind them, leaving domain meaning in informal documentation rather than in any artifact the framework can reason about. We elevate this implicit connection to a first-class formal object through the interpretation function, mapping each label to a formula over the ontology that captures its intended domain meaning. This is not a new engineering step, as safety-critical practice already demands it. Tools like FRET [44] and STIMULUS [45], for instance, requires engineers to provide explicit variable bindings between requirements and system models. Our interpretation plays the same role, grounding observable behaviour in shared domain semantics rather than in system-specific variable names.

Definition 6. Given a TTS $\mathcal{T} = (S, \Sigma_E, \Sigma_D, \to)$ and an ontology K, an interpretation is a function $I : S \cup \Sigma_E \mapsto \mathcal{L}(K)$

Example 2. Continuing Example 1, consider a TTS $\mathcal{T}$ with $\Sigma_E = \{enter\_zone,\ exit\_zone,\ shutoff,\ resume,\ spray\}$ modelling the drone's observable behaviour. The engineer assigns the following interpretations:

$$\begin{aligned} I(spray) &= consumption(current\_hectare) \leq 50 \\ I(shutoff) &= total\_consumption \leq battery\_capacity \\ &\quad \wedge\ covered(current\_hectare) \\ I(exit\_zone) &= covered(current\_hectare) \end{aligned}$$

The first captures that a *spray* transition is fired only when the energy needed for the current hectare remains within the per-hectare budget. The second captures that *shutoff* is meaningful only when cumulative consumption has not exceeded battery capacity and the current hectare has been fully covered. The third captures that *exit_zone* is fired upon completion of coverage of the current hectare.

For composite systems, each component carries its own interpretation over its local alphabet, with alphabets pairwise disjoint. We write $\mathcal{I} = \{I_1, \ldots, I_n\}$ where $\mathrm{dom}(I_i) \cap \mathrm{dom}(I_j) = \emptyset$ for all $i \neq j$. Accordingly, we can now define the domain knowledge simply as the pair $\Phi = (K, \mathcal{I})$.

As development progresses, engineers refine domain knowledge by discovering tighter constraints or new relationships. For example, a datasheet may reveal a lower battery rating, or field measurements may show higher-than-expected consumption. We wish to formalize this intuition with the following definition, which captures the idea of one domain knowledge being more precise than another.

Definition 7. Let $\Phi' = (K', \mathcal{I}')$ and $\Phi = (K, \mathcal{I})$. We say Φ' *refines* Φ, written $\Phi' \sqsubseteq \Phi$, iff:

I) $(S', F', R') \supseteq (S, F, R)$, (vocabulary extension)
II) $\Delta' \models \Delta$ (axiom strengthening)

Condition I allows the refined ontology to introduce new vocabulary, while Condition II requires its axioms to entail all prior ones. In this way, existing knowledge is never contradicted, only strengthened.

Example 3. Continuing Example 1, suppose that after field deployment the engineering team refines their domain model as new information about motor dynamics, battery limits, and energy consumption becomes available. The refined ontology K' (i) extends the vocabulary with a type $Motor$ and function $rpm : Motor \to \mathbb{R}$ (Condition I), (ii) strengthens the axioms by updating $battery\_capacity = 450$ and adding $\forall h : Hectare.\ consumption(h) \leq 100$ (Condition II).

The following theorem establishes a desirable property of refinement, namely its conservativity: every model of K' is also a model of K, and every admissible letter under Φ' remains admissible under Φ. As a result, engineers can safely refine their domain knowledge without invalidating previously established results, making iterative development feasible.

Theorem 1. *If $\Phi' \sqsubseteq \Phi$, then:*

I) every model of K' is also a model of K,
II) every admissible event under Φ' is admissible under Φ,

Proof. Assume $\Phi' \sqsubseteq \Phi$.

(I) Let $\mathcal{M} \models \Delta'$. Since $(S, F, R) \subseteq (S', F', R')$ by condition I, the reduct $\mathcal{M}|_{(S,F,R)}$ is a well-defined structure over the signature of K. Since $\Delta' \models \Delta$ by condition II, we have $\mathcal{M} \models \Delta$, and $\mathcal{M}|_{(S,F,R)} \models \Delta$. Hence $\mathcal{M}$ is a model of K.
(II) Let $a \in \mathrm{dom}(I'_i)$ for some $I'_i \in \mathcal{I}'$ be admissible under Φ', i.e., there exists $\mathcal{M} \models \Delta'$ such that $\mathcal{M} \models I'_i(a)$. By (I), $\mathcal{M} \models \Delta$, so a is admissible under Φ. □

VI. Semantic Alignment between PT and DT

As already established in Section II, we aim at establishing semantic alignment at the view level, represented as TAs (cf. Section IV). Since the two twins serve different purposes and may be authored and maintained by different stakeholders, their views typically include behavior that are specific to each twin and do not need to be reflected in the other, e.g. internal physical events on the PT side, logging or external API calls on the DT side. Semantic alignment, therefore, amounts to establish that the PT and DT are semantically coherent only on the behaviors relevant to monitoring and decision-making. In other words, semantic alignment shall guarantee that the PT and DT exhibit matching behavior for all aspects relevant to the aforementioned tasks, while preserving a consistent semantic interpretation of such behavior.

To formalize these guarantees, we extend WTB (cf. Section IV) by requiring that matched states and transitions carry semantically equivalent interpretations under the shared domain knowledge $\Phi = (K, \mathcal{I})$ (cf. Section V). This assumption is consistent with established DT practice [18], [20] and with interoperability frameworks for Industry 4.0 ecosystems based on the Asset Administration Shell (AAS), which employ shared semantic models and ontologies to enable integration and capability discovery among independently developed twins [46], [29], [31]. Similar principles underpin federated data-space initiatives such as Gaia-X [14].

Definition 8. Let $V_P = (S^P, \Sigma_E^P, \Sigma_D^P, \to^P)$ and $V_D = (S^D, \Sigma_E^D, \Sigma_D^D, \to^D)$ be TTS representations of the PT and DT views, respectively, and let $A = \mathrm{dom}(I_P) \subseteq \Sigma_E^P$ and $B = \mathrm{dom}(I_D) \subseteq \Sigma_E^D$ be the ontologically relevant sub-alphabets of the PT and DT, respectively. We say V_P and V_D are semantically aligned under $\Phi = (K, \{I_P, I_D\})$, written as $V_P \overset{K}{\sim} V_D$, if there exists a relation $\overset{K}{\sim} \subseteq S^P \times S^D$ such that $(s_0^{V_P}, s_0^{V_D}) \in \overset{K}{\sim}$ and for every $(p, d) \in \overset{K}{\sim}$:

I) $\Delta \models I_P(p) \leftrightarrow I_D(d)$ (state consistency)

II) $\forall a \in A,\ p'$ with $p \overset{a}{\Longrightarrow} p',\ \exists a' \in B,\ d' : d \overset{a'}{\Longrightarrow} d' \wedge (p', d') \in \overset{K}{\sim} \wedge\, \Delta \models I_P(a) \leftrightarrow I_D(a')$ (PT-to-DT)

III) $\forall a' \in B,\ d'$ with $d \overset{a'}{\Longrightarrow} d',\ \exists a \in A,\ p' : p \overset{a}{\Longrightarrow} p' \wedge (p', d') \in \overset{K}{\sim} \wedge\, \Delta \models I_P(a) \leftrightarrow I_D(a')$ (DT-to-PT)

IV) $\forall \delta \in \Sigma_D,\ p'$ with $p \xrightarrow{\delta} p',\ \exists d'$ reachable from d by $\overset{\varepsilon}{\Longrightarrow}$ with elapsed time δ such that $(p', d') \in \overset{K}{\sim}$, and symmetrically (delay consistency)

Condition I ensures that matched states carry equivalent domain-level meaning under K: the PT and DT agree on the facts of the world at every corresponding point in their execution. Conditions II and III extend the standard bisimulation clauses by requiring ontologically equivalent labels to match, rather than syntactically identical ones: a PT transition on $a \in A$ must be matched by a DT transition on some $a' \in B$ with $\Delta \models I_P(a) \leftrightarrow I_D(a')$, and vice versa. Labels outside A and B are treated as τ-transitions, internal and invisible to the alignment, while Condition IV ensures timing is preserved across matched pairs. Standard syntactic bisimulation arises as the special case $a = a'$ and $I_P(a) = I_D(a)$.

Definition 8, makes V_P and V_D effectively equivalent in terms of their observable behavior and semantic interpretation under Φ. This makes the two views semantically indistinguishable and interchangeable for domain reasoning. Much of the DT literature assumes a high-fidelity correspondence between PT and DT, often described as continuous mirroring or bidirectional synchronization of state and behavior [47], [**?**], [7]. Such assumptions conceptually align with bisimulation-like behavioral correspondence. However, in cases where full correspondance is not required, Defintion 8 can be relaxed to a simulation relation, e.g. by neglecting Condition III. In this case, we write $V_P \preccurlyeq_\Phi V_D$.

Algorithm 1: Semantic Alignment Check

Input: $V_P,\ V_D,\ \Phi = (K, \{I_P, I_D\})$
Output: `True` iff $V_P \overset{K}{\sim} V_D$

$E \leftarrow \{(a, a') \in A \times B \mid \Delta \models I_P(a) \leftrightarrow I_D(a')\}$
$\overset{K}{\sim} \leftarrow \{(p, d) \in S^P \times S^D \mid \Delta \models I_P(p) \leftrightarrow I_D(d)\}$
repeat
  $\overset{K'}{\sim} \leftarrow \overset{K}{\sim}$
  foreach $(p, d) \in \overset{K}{\sim}$ **do**
    foreach $a \in A,\ p \overset{a}{\Longrightarrow} p'$ **do** `// Cond. II`
      if $\nexists (a', d') : (a, a') \in E,\ d \overset{a'}{\Longrightarrow} d',\ (p', d') \in \overset{K}{\sim}$ **then** $\overset{K'}{\sim} \leftarrow \overset{K'}{\sim} \setminus \{(p, d)\}$
    foreach $a' \in B,\ d \overset{a'}{\Longrightarrow} d'$ **do** `// Cond. III`
      if $\nexists (a, p') : (a, a') \in E,\ p \overset{a}{\Longrightarrow} p',\ (p', d') \in \overset{K}{\sim}$ **then** $\overset{K'}{\sim} \leftarrow \overset{K'}{\sim} \setminus \{(p, d)\}$
    foreach $\delta,\ p \xrightarrow{\delta} p'$ **do** `// Cond. IV, PT`
      if $\nexists d' : d \overset{\varepsilon}{\Longrightarrow} d'$ *with elapsed* δ, $(p', d') \in \overset{K}{\sim}$ **then** $\overset{K'}{\sim} \leftarrow \overset{K'}{\sim} \setminus \{(p, d)\}$
    foreach $\delta,\ d \xrightarrow{\delta} d'$ **do** `// Cond. IV, DT`
      if $\nexists p' : p \overset{\varepsilon}{\Longrightarrow} p'$ *with elapsed* δ, $(p', d') \in \overset{K}{\sim}$ **then** $\overset{K'}{\sim} \leftarrow \overset{K'}{\sim} \setminus \{(p, d)\}$
  $\overset{K}{\sim} \leftarrow \overset{K'}{\sim}$
until $\overset{K}{\sim}$ *stabilises*
return `True` *iff* $(s_0^{V_P}, s_0^{V_D}) \in \overset{K}{\sim}$, else `False`

The syntactic representation of views as TAs makes Definition 8 directly amenable to fixpoint computation, given in Algorithm 1. The algorithm follows the standard greatest fixpoint characterization of bisimulation [37], adapted to the timed setting via zone-graph exploration [38], [39], with two extensions that operationalize Definition 8 directly. First, the label equivalence relation E is precomputed once via SMT entailment checks over Φ (line 1), amortizing the SMT cost across all subsequent state-pair iterations. The loop then uses E in place of syntactic label equality, requiring at most $|A|\cdot|B|$ solver calls in total. Second, the relation $\overset{K}{\sim}$ is seeded with all state pairs satisfying state consistency (line 2) rather than the full Cartesian product, which prunes the search space. The loop then iteratively removes pairs that violate any of Conditions II–IV, until stabilisation. Termination is guaranteed by the finiteness of the zone abstraction of any TA [38], [39]. Since the relation can only shrink, stabilisation is reached in at most $|S^P|\cdot|S^D|$ iterations. The algorithm yields the largest relation satisfying Definition 8, and returns `True` iff the initial state pair $(s_0^{V_P}, s_0^{V_D})$ belongs to it.

VII. Correctness Guarantees and Practical Implications of Semantic Alignment

Semantic alignment, as defined in Definition 8, is the fundamental correctness criterion of our framework, as it captures the intended semantic relationship between the PT and DT views. In this section we establish two key properties that are guaranteed by semantic alignment, i.e. data and behavioral consistency, as well as their practical implications for the design and maintenance of twins.

The first key property guaranteed by semantic alignment is in the form of semantic consistency of observed data between the PT and DT. More specifically, it guarantees that any domain fact derivable from the PT's state at any point in the execution is equivalently derivable from the matching DT state, and vice versa. We formalize it in the following theorem.

Theorem 2. *Let $V_P \overset{K}{\sim} V_D$ under $\Phi = (K, \{I_P, I_D\})$. Then for every $(p,d) \in \overset{K}{\sim}$ and every $f \in \mathcal{L}(K)$: $\exists \mathcal{M} \models \Delta : \mathcal{M} \models I_P(p) \wedge f \iff \exists \mathcal{M} \models \Delta : \mathcal{M} \models I_D(d) \wedge f$*

Proof. By Definition 8, condition I, for every $(p,d) \in \overset{K}{\sim}$ we have $\Delta \models I_P(p) \leftrightarrow I_D(d)$. Therefore for any model $\mathcal{M} \models \Delta$, $\mathcal{M} \models I_P(p)$ iff $\mathcal{M} \models I_D(d)$. Hence for any $f \in \mathcal{L}(K)$, the existence of a model witnessing $I_P(p) \wedge f$ is equivalent to the existence of a model witnessing $I_D(d) \wedge f$. □

Theorem 2 establishes that this agreement holds at design time, across all reachable matched states, without recourse to observed operational traces. This translates to sensor readings, thresholds, and physical quantities being interpreted identically by both twins, a strong form of data consistency, as the agreement is not on raw data values, which the PT and DT may encode differently, nor on the labels used by either model, which need not coincide in federated settings, but on the semantic content of the state of the world under Φ.

Example 4. Consider again the drone of Example 1. Suppose the PT reaches a state p after spraying three hectares, and that $I_P(p) = (\mathit{total\_consumption} = 150)$. By Definition 8, the matched DT state d satisfies $\Delta \models I_P(p) \leftrightarrow I_D(d)$, so any model of Δ in which the PT has consumed 150 units also has the DT in a state where the same is true, and vice versa. From this, every domain-level fact derivable under Δ, like for example, that $\mathit{battery\_capacity} - \mathit{total\_consumption} = 350$, or that the drone has sufficient charge to cover the remaining seven hectares, holds at p if and only if it holds at d.

The second property guaranteed by semantic alignment is behavioral consistency: any requirement verified on the PT automatically holds on the DT, and vice versa. Formally, we establish this by showing that the PT and DT satisfy the same TCTL properties over their ontologically relevant alphabets. Definition 8 allows properties to be transferred between views whose labels differ, as long as they are semantically equivalent under Φ. This makes behavioral consistency establishable between independently developed twins that share no common label vocabulary, which is precisely the setting of federated architectures.

To transfer properties between the PT and DT views, we need a way to identify which PT and DT labels carry the same domain-level meaning under the shared ontology. We call two labels $a \in \mathrm{dom}(I_P)$ and $b \in \mathrm{dom}(I_D)$ equivalent under Δ, written $a \equiv_\Delta b$, if $\Delta \models I_P(a) \leftrightarrow I_D(b)$. This groups labels into equivalence classes and we write $[\ell]$ for the class of ℓ.

Definition 9. Let $V_P \overset{K}{\sim} V_D$. The induced label translation is the partial function $\lambda : A/{\equiv_\Delta} \rightharpoonup B/{\equiv_\Delta}$ defined by $\lambda([a]) = [a'] \iff \Delta \models I_P(a) \leftrightarrow I_D(a')$.

Intuitively, λ is the dictionary that bridges the two independently developed vocabularies: it maps each PT label to the DT label that means the same thing under Φ. It follows from Definition 8 that λ is well-defined and covers all labels reachable under the alignment relation.

Applying λ as a substitution to a TCTL property φ yields $\lambda(\varphi)$, i.e. the same property expressed in the DT vocabulary. Theorem 3 uses this to show that any property verified on the PT holds on the DT under $\lambda(\varphi)$, and vice versa.

Theorem 3. *Let $V_P \overset{K}{\sim} V_D$ and let λ be the induced label translation. Then for every TCTL formula φ over $\mathrm{dom}(I_P)$: $V_P \models \varphi \iff [\![V_D]\!] \models \lambda(\varphi)$ and symmetrically, for every ψ over $\mathrm{dom}(I_D)$: $V_D \models \psi \iff [\![V_P]\!] \models \lambda^{-1}(\psi)$*

Proof. We show the forward direction; the reverse is symmetric. By Definition 8, $\overset{K}{\sim}$ is a WTB between V_P and V_D on the sub-alphabets $\mathrm{dom}(I_P)$ and $\mathrm{dom}(I_D)$ modulo the relabeling induced by λ. Let $\sigma_\lambda(V_D)$ denote V_D with each $a' \in \mathrm{dom}(I_D)$ renamed to a fixed representative of $\lambda^{-1}([a'])$ when defined. Then V_P and $\sigma_\lambda(V_D)$ are weak timed bisimilar in the syntactic sense over $\mathrm{dom}(\lambda)$, and TCTL preservation under weak timed bisimulation yields $[\![V_P]\!] \models \varphi \iff [\![\sigma_\lambda(V_D)]\!] \models \varphi$. Unfolding σ_λ gives the claim. □

Example 5. Consider again the drone of Example 1. Suppose the PT engineer has verified the safety property $\varphi = \mathbf{AG}(\mathit{spray} \rightarrow \mathbf{AF}\, \mathit{shutoff})$ on V_P. The DT models the same physical events under different labels and domain predicates. In the DT view, the spray condition is expressed in terms of the remaining energy budget, i.e., $I_D(\mathit{apply}) = \mathit{battery\_capacity} - \mathit{total\_consumption} - \mathit{consumption}(\mathit{current\_hectare}) \geq \mathit{battery\_capacity} - 50$.

Similarly, the shutoff condition is expressed through residual energy and coverage, i.e. $I_D(\mathit{power\_down}) = \mathit{battery\_capacity} - \mathit{total\_consumption} \geq 0 \wedge \mathit{covered}(\mathit{current\_hectare})$. Although syntactically different from $I_P(\mathit{spray})$ and $I_P(\mathit{shutoff})$, these predicates are equivalent under Δ. The arithmetic axioms over $\mathbb{R}$ together with $\mathit{battery\_capacity} = 500$ and the non-negativity of $\mathit{consumption}$ entail $\Delta \models I_P(\mathit{spray}) \leftrightarrow I_D(\mathit{apply})$ and $\Delta \models I_P(\mathit{shutoff}) \leftrightarrow I_D(\mathit{power\_down})$. By Definition 9, the induced label translation maps $\lambda([\mathit{spray}]) = [\mathit{apply}]$ and $\lambda([\mathit{shutoff}]) = [\mathit{power\_down}]$. Theorem 3 therefore yields that V_D satisfies $\lambda(\varphi) = \mathbf{AG}(\mathit{apply} \rightarrow \mathbf{AF}\, \mathit{power\_down})$.

As additional domain knowledge becomes available during system development or deployment, the ontology can be updated to reflect it. A key advantage of our framework is that the design-time guarantees of semantic alignment is not invalidated by refinements of the ontology, as we establish in the following theorem.

Theorem 4. *Let Φ'and Φ be two domain knowledge instances such that $\Phi' \sqsubseteq \Phi$, if $V_P \overset{K}{\sim} V_D$ then $V_P \overset{K'}{\sim} V_D$.*

Proof. Semantic alignment requires that for every event e, $\Delta \models I_P(e) \leftrightarrow I_D(e)$. Since $\Delta' \models \Delta$, it follows that $\Delta' \models I_P(e) \leftrightarrow I_D(e)$. Therefore the alignment condition continues to hold under K'. □

Consequently, ontology refinements that conservatively extend domain knowledge do not invalidate previously established alignment results or property-transfer guarantees.

As established in Section II, any implementation of the PT and DT must refine their respective views, which is standard practice in MBSE-based design. One the one hand, the implemented DT software must refine the DT view in a way that realizes the specified DT behavior, which is achievable by construction since the DT is software under the engineer's control, formally $D \sim V_D$. On the other hand, the deployed physical system can not be expected to reproduce the exact behavior of the PT view, as the view necessarily abstracts from the full complexity of the underlying physical system. Instead, it is typically required only to refine its model by avoiding behaviors that the model forbids, formally $P \preccurlyeq V_P$. Under these conditions, we can show that the the implemented and deployed system inherits the semantic consistency guarantees of the views for all domain facts and safety properties.

Theorem 5. *Under the framework, the deployed PT and DT inherit the data-consistency and safety property guarantees established by the semantic alignment of their views*

Proof. By conditions (i) and (ii) and Definition 8, we obtain the refinement chain $P \preccurlyeq V_P \overset{K}{\sim} V_D \sim D$ Theorem 2 and Theorem 3 establish the corresponding guarantees for the views. Data facts are preserved pointwise through the chain, while safety TCTL properties are preserved under simulation from V_P to P. The claim follows. □

Note that Theorem 5 also hold for $V_P \preccurlyeq_K V_D$.

Theorem 5 has significant practical implications for the verification of deployed systems. First, it establishes universal data consistency for all reachable states and for all facts expressible in the ontology: any domain-level fact that holds for the deployed PT at any point in its execution also holds for the deployed DT, and vice versa. Second, it guarantees behavioral consistency for all safety properties expressible over the ontologically relevant alphabets, since simulation preserves safety [37]. These guarantees establish the DT as a sound representation of the deployed PT.

Firstly, any safety property verified on the DT is guaranteed to hold for the deployed PT, and any ontology-expressible fact derived from the DT is guaranteed to hold for the PT. This has direct implications for runtime monitoring, which is the predominant mechanism currently used to assess PT/DT [15], [33], [17], [18]. Runtime monitors observe finite prefixes of PT executions and check whether selected data-level or behavioral properties hold over the observed traces. Theorem 5 shows that, for properties expressible over the ontologically relevant alphabets, these verdicts are already guaranteed by design-time alignment verification.

The theorem also enables predictive use of the DT. Any prediction made by the DT about the future behavior of the PT is guaranteed to respect all safety properties established at the model level, and any domain-level prediction is guaranteed to remain consistent with the deployed PT. The DT can therefore serve as a sound predictor of the PT's future behavior within with respect to the ontologically relevant alphabets. In practice, this enables safe what-if analysis, predictive decision support, and operational planning, as engineers can explore alternative future behaviors without risking violations of verified safety constraints.

More broadly, Theorem 5 has implications for the cost and scalability of twin design and maintenance. Establishing semantic alignment at design time is a one-off effort that yields guarantees that hold universally for the deployed system and for all ontology-expressible properties. Furthermore, it relies only on artifacts that are already available in typical DT and MBSE workflows, namely system models [1] and domain ontologies [26], [27], [28], [29], [30], [31], [32], and does not require additional engineering effort beyond current practice. Compared to runtime monitoring, for instance, engineers do not need to construct and maintain dedicated artifacts, such as runtime monitors throughout design and deployment cycles, which is a cost that can be significant given that each monitor typically covers only a limited set of properties. Because alignment is established at the level of views, any implementation or deployment refinement that respects these views

inherits the same guarantees without additional verification effort. This supports scalable iterative development, as new implementations and deployments automatically inherit the guarantees established by the views. It also facilitates interoperability: independently developed twins that refine the same view inherit alignment by construction. In federated settings, stakeholders need only share the relevant abstract views, without exposing implementation details. These shared views therefore act as semantic contracts that enable interoperability while preserving implementation independence.

REFERENCES

[1] INCOSE, *Systems Engineering Handbook: A Guide for System Life Cycle Processes and Activities*, 5th ed., D. D. Walden, G. J. Roedler, K. J. Forsberg, R. D. Hamelin, and T. M. Shortell, Eds. Hoboken, NJ, USA: John Wiley & Sons, 2023.

[2] S. Kugele, P. Obergfell, and E. Sax, "Model-based resource analysis and synthesis of service-oriented automotive software architectures," *Software and Systems Modeling*, vol. 20, no. 6, pp. 1945–1975, 2021.

[3] M. Broy, W. Damm, S. Henkler, K. Pohl, A. Vogelsang, and T. Weyer, "Introduction to the spes modeling framework," in *Model-Based Engineering of Embedded Systems: The SPES 2020 Methodology*. Springer, 2012, pp. 31–49.

[4] EUROCAE Working Group 71, *Software Considerations in Airborne Systems and Equipment Certification*, EUROCAE Std. ED-12C / DO-178C, 2011, european equivalent of DO-178C, harmonized standard for avionics software certification.

[5] RTCA Special Committee 205, *Software Considerations in Airborne Systems and Equipment Certification*, RTCA, Inc. Std. DO-178C / ED-12C, 2011, issued jointly with EUROCAE as ED-12C; guidance for development of safety-critical airborne software.

[6] M. Grieves, "Digital twin: Manufacturing excellence through virtual factory replication," University of Michigan, Tech. Rep., 2005, available online at https://doi.org/10.13140/RG.2.2.26367.02729.

[7] F. Tao, Q. Qi, A. Liu, and A. Kusiak, "Data-driven smart manufacturing," *Journal of Manufacturing Systems*, vol. 48, pp. 157–169, 2018.

[8] R. Minerva, G. M. Lee, and N. Crespi, "Digital twin in the IoT context: A survey on technical features, scenarios, and architectural models," *Proceedings of the IEEE*, vol. 108, no. 10, pp. 1785–1824, 2020.

[9] J. Woodcock, C. Gomes, H. D. Macedo, and P. G. Larsen, "Uncertainty quantification and runtime monitoring using environment-aware digital twins," in *Leveraging Applications of Formal Methods, Verification and Validation (ISoLA 2020), Part IV*, ser. LNCS, vol. 12479. Springer, 2021, pp. 72–87.

[10] M. Liu, S. Fang, H. Dong, and C. Xu, "Review of digital twin about concepts, technologies, and industrial applications," *Journal of Manufacturing Systems*, vol. 58, pp. 346–361, 2021.

[11] F. Tao, M. Zhang, Y. Liu, and A. Y. C. Nee, "Digital twin driven prognostics and health management for complex equipment," *CIRP Annals*, vol. 67, no. 1, pp. 169–172, 2018.

[12] J. Mertens, S. Klikovits, F. Bordeleau, J. Denil, and Ø. Haugen, "Continuous evolution of digital twins using the DarTwin notation," *Software and Systems Modeling*, 2024.

[13] D. Lehner, J. Zhang, J. Pfeiffer, S. Sint, A.-K. Splettstößer, M. Wimmer, and A. Wortmann, "Model-driven engineering for digital twins: a systematic mapping study," *Software and Systems Modeling*, 2025.

[14] GAIA-X European Association for Data and Cloud AISBL, "Gaia-x architecture document," https://gaia-x.eu/pdf/Gaia-X_Architecture_Document_2103.pdf, 2021.

[15] P. Muñoz, J. Troya, and A. Vallecillo, "Towards measuring digital twins fidelity at runtime," in *Proceedings of the ACM/IEEE 27th International Conference on Model Driven Engineering Languages and Systems*, ser. MODELS Companion '24. New York, NY, USA: Association for Computing Machinery, 2024, p. 507–512. [Online]. Available: https://doi.org/10.1145/3652620.3688267

[16] D. Lehner, S. Sint, M. Vierhauser, W. Narzt, and M. Wimmer, "Aml4dt: A model-driven framework for developing and maintaining digital twins with automationml," in *2021 26th IEEE International Conference on Emerging Technologies and Factory Automation (ETFA)*, IEEE. Zaragoza, Spain: IEEE, 2021, pp. 1–8.

[17] E. Kamburjan, C. C. Din, R. Schlatte, S. L. T. Tarifa, and E. B. Johnsen, "Twinning-by-construction: Ensuring correctness for self-adaptive digital twins," in *Leveraging Applications of Formal Methods, Verification and Validation. Verification Principles: 11th International Symposium, ISoLA 2022, Rhodes, Greece, October 22–30, 2022, Proceedings, Part I*. Berlin, Heidelberg: Springer-Verlag, 2022, p. 188–204. [Online]. Available: https://doi.org/10.1007/978-3-031-19849-6_12

[18] S. Gil, E. Kamburjan, P. Talasila, and P. G. Larsen, "An architecture for coupled digital twins with semantic lifting," *Software and Systems Modeling*, vol. 24, no. 5, pp. 1379–1404, 2025. [Online]. Available: https://doi.org/10.1007/s10270-024-01221-w

[19] X. Ma, Q. Qi, and F. Tao, "An ontology-based data-model coupling approach for digital twin," *Robot. Comput.-Integr. Manuf.*, vol. 86, no. C, Apr. 2024. [Online]. Available: https://doi.org/10.1016/j.rcim.2023.102649

[20] E. Karabulut, S. F. Pileggi, P. Groth, and V. Degeler, "Ontologies in digital twins: A systematic literature review," *Future Generation Computer Systems*, vol. 153, pp. 442–456, 2024. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S0167739X23004739

[21] E. Glaessgen and D. Stargel, "The digital twin paradigm for future nasa and u.s. air force vehicles," in *53rd AIAA/ASME/ASCE/AHS/ASC Structures, Structural Dynamics and Materials Conference*. Honolulu, Hawaii: American Institute of Aeronautics and Astronautics (AIAA), 04 2012.

[22] D. M. Botín-Sanabria, A.-S. Mihaita, R. E. Peimbert-García, M. A. Ramírez-Moreno, R. A. Ramírez-Mendoza, and J. d. J. Lozoya-Santos, "Digital twin technology challenges and applications: A comprehensive review," *Remote Sensing*, vol. 14, no. 6, p. 1335, 2022.

[23] S. Mihai, M. Yaqoob, D. V. Hung, W. Davis, P. Towakel, M. Raza, M. Karamanoglu, B. Barn, D. Shetve, R. V. Prasad *et al.*, "Digital twins: A survey on enabling technologies, challenges, trends and future prospects," *IEEE Communications Surveys & Tutorials*, vol. 24, no. 4, pp. 2255–2291, 2022.

[24] L. Brodo, G. Scalora, and S. Henkler, "Towards a digital twin framework for secure and efficient cyber-physical transportation systems," in *Proceedings of the 1st Workshop on Modeling and Verification for Secure and Performant Cyber-Physical Systems*, ser. MoVe4SPS '25. New York, NY, USA: Association for Computing Machinery, 2025. [Online]. Available: https://doi.org/10.1145/3735948.3736156

[25] C. Steinmetz, G. N. Schroeder, A. Sulak, K. Tuna, A. Binotto, A. Rettberg, and C. E. Pereira, "A methodology for creating semantic digital twin models supported by knowledge graphs," in *2022 IEEE 27th International Conference on Emerging Technologies and Factory Automation (ETFA)*, IEEE. Stuttgart, Germany: IEEE, 2022, pp. 1–7.

[26] C. Boje, A. Guerriero, S. Kubicki, and Y. Rezgui, "Towards a semantic construction digital twin: Directions for future research," *Automation in Construction*, vol. 114, p. 103179, 2020.

[27] M. Elhajj, "Semantic foundations for digital twins: the contribution of ontological analysis," *Frontiers in Computer Science*, vol. Volume 8 - 2026, 2026. [Online]. Available: https://www.frontiersin.org/journals/computer-science/articles/10.3389/fcomp.2026.1757450

[28] B. Oakes, C. Gomes, E. Kamburjan, G. Abbiati, E. Ecem Bas, and S. Engelsgaard, "Towards ontological service-driven engineering of digital twins," in *Proceedings of the ACM/IEEE 27th International Conference on Model Driven Engineering Languages and Systems*, ser. MODELS Companion '24. New York, NY, USA: Association for Computing Machinery, 2024, p. 464–469. [Online]. Available: https://doi.org/10.1145/3652620.3688261

[29] I. Grangel-González, L. Halilaj, G. Coskun, S. Auer, D. Collarana, and M. Hoffmeister, "Towards a semantic administrative shell for industry 4.0 components," in *Proceedings of the IEEE 10th International Conference on Semantic Computing (ICSC)*. IEEE, 2016, pp. 230–237.

[30] S. R. Bader and M. Maleshkova, "The semantic asset administration shell," *arXiv preprint arXiv:1909.00690*, 2019.

[31] J. Arm, T. Benešl, P. Marcon, Z. Bradáč, and T. Schröder, "Automated design and integration of asset administration shells in components of industry 4.0," *Sensors*, vol. 21, no. 6, p. 2004, 2021.

[32] F. Farias *et al.*, "Dtag: A methodology for aggregating digital twins using the wotdt ontology," *Applied Sciences*, vol. 14, no. 13, p. 5960, 2024. [Online]. Available: https://doi.org/10.3390/app14135960

[33] M. G. Seok, W. J. Tan, W. Cai, and D. Park, "Digital-twin consistency checking based on observed timed events with unobservable transitions

in smart manufacturing," *IEEE Transactions on Industrial Informatics*, vol. 19, no. 4, pp. 6208–6219, 2023.

[34] L. Huang, L. R. Varshney, and K. E. Willcox, "Formal verification of digital twins with tla and information leakage control," *arXiv preprint arXiv:2411.18798*, 2024. [Online]. Available: https://arxiv.org/abs/2411.18798

[35] H. Liu, X. Song, G. Gao, H. Zhang, Y.-S. Liu, and M. Gu, "Modeling and validating temporal rules with semantic petri net for digital twins," *Advanced Engineering Informatics*, vol. 58, p. 102099, 2023. [Online]. Available: https://doi.org/10.1016/j.aei.2023.102099

[36] R. Alur and D. L. Dill, "A theory of timed automata," *Theoretical Computer Science*, vol. 126, no. 2, pp. 183–235, 1994. [Online]. Available: https://www.sciencedirect.com/science/article/pii/0304397594900108

[37] C. Baier and J.-P. Katoen, *Principles of Model Checking*. MIT Press, 2008.

[38] R. Alur, C. Courcoubetis, and D. Dill, "Model-checking in dense real-time," *Information and Computation*, vol. 104, no. 1, pp. 2–34, 1993. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S0890540183710242

[39] P. Bouyer, "An introduction to timed automata," in *Formal Modeling and Analysis of Timed Systems*, ser. Lecture Notes in Computer Science. Springer, 2006, vol. 3829, pp. 1–26.

[40] C. Barrett and C. Tinelli, *Satisfiability Modulo Theories*. Springer, 2018.

[41] N. F. Noy and D. L. McGuinness, "Ontology development 101: A guide to creating your first ontology," 2001, technical Report KSL-01-05.

[42] R. Stevens, C. Goble, and S. Bechhofer, "Ontology-based knowledge representation for bioinformatics," *Briefings in Bioinformatics*, vol. 1, no. 4, pp. 398–414, 2000.

[43] A. Hogan, E. Blomqvist, M. Cochez, C. d'Amato, G. d. Melo, C. Gutierrez, S. Kirrane, J. E. L. Gayo, R. Navigli, S. Neumaier *et al.*, "Knowledge graphs," *ACM Computing Surveys*, vol. 54, no. 4, pp. 1–37, 2021.

[44] D. Giannakopoulou, T. Pressburger, A. Mavridou, and J. Schumann, "Automated formalization of structured natural language requirements," *Information and Software Technology*, vol. 137, p. 106590, 2021.

[45] Dassault Systèmes, "Catia stimulus," https://www.3ds.com/products/catia/stimulus, Dassault Systèmes, 2026, accessed: 2026-04-07. [Online]. Available: https://www.3ds.com/products/catia/stimulus

[46] I. Grangel-González, L. Halilaj, and S. Lohmann, "Semantic interoperability of digital twins: Ontology-based capability checking in aas modeling framework," in *Proceedings of the IEEE International Conference on Industrial Informatics*. IEEE, 2023.

[47] W. Kritzinger, M. Karner, G. Traar, J. Henjes, and W. Sihn, "Digital twin in manufacturing: A categorical literature review and classification," *IFAC-PapersOnLine*, vol. 51, no. 11, pp. 1016–1022, 2018.